Bullet

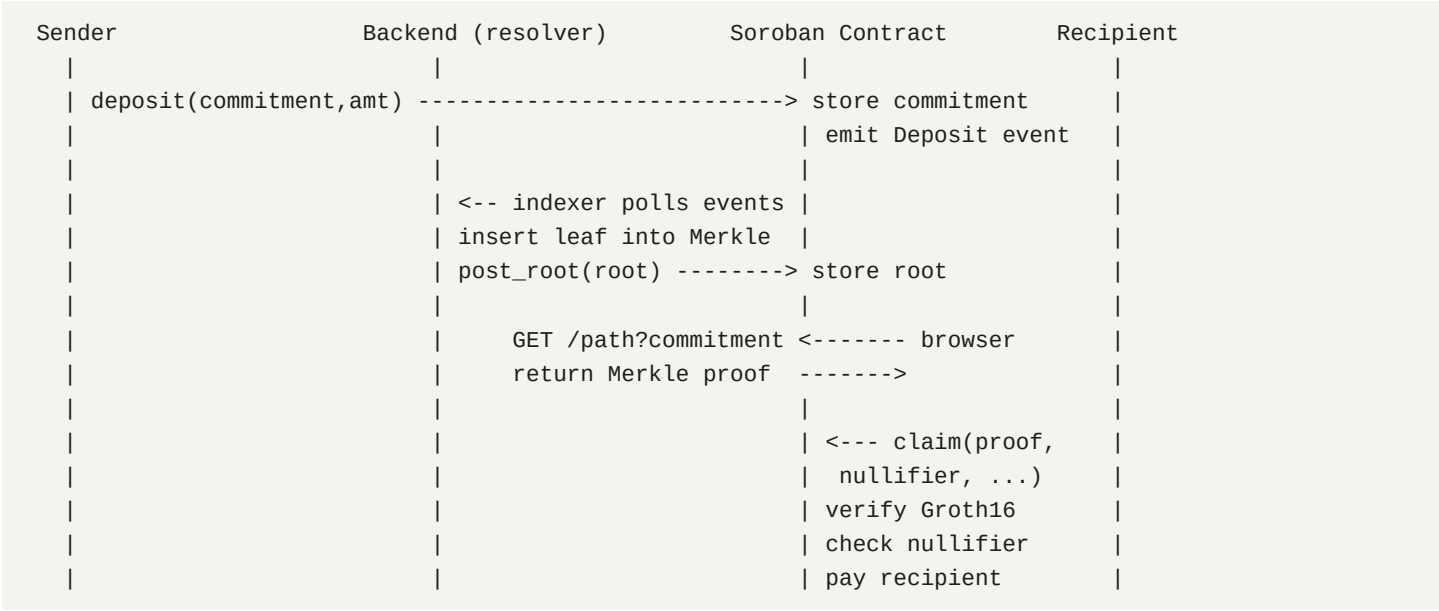
ZK-Private Payment Rail on Stellar
Technical Architecture

Groth16 / BLS12-381 / Poseidon Merkle / Soroban

July 2026

1. High-Level Flow

Bullet lets a sender pay USDC, XLM, or USDT to a social handle (X) or email instead of a wallet address, while hiding the sender-recipient link on-chain. No on-chain field links a deposit tx to a claim tx.



2. Proof System: Groth16 over BLS12-381

Soroban provides native bls12_381 host functions (pairing_check, g1_msm, g1_mul, fr_sub, etc.). A benchmark showed a 4-pair pairing check + 5-point IC MSM uses ~70.66% of the 100M CPU instruction budget per tx, leaving ~29% headroom. Fee per verify: ~0.006 XLM. PLONK/off-chain fallbacks are not needed.

The Circom Circuit (circuits/src/claim.circom)

Depth-20 Poseidon-Merkle membership + nullifier derivation. 5 public inputs, 3 private inputs.

Public Inputs (order matches derive_public_inputs in lib.rs)

```
[root, nullifier, recipientDigest, amount, tokenId]
```

Private Inputs

```
secret, pathElements[20], pathIndices[20]
```

Proof Statement

The prover knows "secret" such that:

- **1.** nullifier = Poseidon(secret) -- arity-1 for domain separation
- **2.** commitment = Poseidon(secret, recipientDigest, amount, tokenId) -- arity-4
- **3.** commitment is a leaf in the Merkle tree with root 'root', proven via 20 levels of Poseidon(left, right) hashing with the path siblings

Why Each Public Input Is Bound

- **root:** proves membership in a contract-known tree state
- **nullifier:** prevents double-spend (contract stores used nullifiers)
- **recipientDigest:** prevents front-running (can't substitute your address)
- **amount:** prevents claiming a different amount than deposited
- **tokenId:** prevents cross-token drain (depositing USDC, claiming XLM)

3. Poseidon Hash Function

JS implementation over BLS12-381 scalar field (r = 52435875175126190479447740508185965837690552500527637822603658699938581184513) using ffjavascript ZqField. Optimized round constants from poseidon_constants_opt.json. x^5 S-box, 8 full rounds + variable partial rounds (57 for arity-2, 56 for arity-1).

Same code runs in both backend (backend/src/poseidon.ts) and browser (frontend/src/lib/poseidon.ts).

Important caveat: uses circomlib BN254 round constants compiled over BLS12-381 (-p bls12381). Not canonical BLS12-381 Poseidon. Works because circom compiles the same arithmetic constraints over whatever field you target. Acceptable for hackathon, replace with native BLS12-381 Poseidon for production.

4. Merkle Tree (Off-Chain, Option B)

The tree is NOT on-chain. This is the trust seam of the system.

- **Step 1:** Indexer (backend/src/indexer.ts) polls Soroban RPC for deposit events
- **Step 2:** Confirmed on-chain deposits get inserted as leaves into an in-memory sparse Merkle tree (backend/src/tree.t depth 20)
- **Step 3:** Leaves persist in Postgres (merkle_store.ts) so the tree survives backend redeploys
- **Step 4:** After inserting new leaves, the indexer calls post_root(root) on the contract (admin-signed tx)
- **Step 5:** Contract stores roots in a ring buffer of 64 slots. Old roots get evicted

Leaf Value

```
Poseidon(secret, recipientDigest, amount, tokenId)  as decimal string
```

Zero Hashes

```
zeroHashes[0] = "0"
zeroHashes[i+1] = Poseidon(zeroHashes[i], zeroHashes[i])
```

5. Browser-Side Proving Flow

File: frontend/src/lib/prove_browser.ts

- **1.** Load claim.wasm + claim.zkey from /circuits/ (cached after first load)
- **2.** Compute commitment = Poseidon(secret, recipientDigest, amount, tokenId)
- **3.** Fetch Merkle path from resolver: GET /path?commitment=<decimal>
- **4.** Compute nullifier = Poseidon(secret). The secret never leaves the tab
- **5.** Run snarkjs.groth16.fullProve(inputs, wasm, zkey) in-browser WASM (~15-30s)
- **6.** Encode proof points as big-endian hex bytes for Soroban

Proof Output Encoding

```
proof_a:    G1 point, 96 bytes  (2 x 48-byte coordinates)
proof_b:    G2 point, 192 bytes (2 x 2 x 48, Fp2 coord swap: [c1,c0] for Soroban)
proof_c:    G1 point, 96 bytes
nullifier:  Fr, 32 bytes big-endian
root:       Fr, 32 bytes big-endian
```

6. On-Chain Verification

File: contracts/zeekpay/src/verifier.rs

Standard Groth16 verify using Soroban BLS12-381 host functions. The verification equation:

```
e(A,B) = e(alpha,beta) * e(L,gamma) * e(C,delta)
```

Rearranged into a single pairing_check call:

```
pairing_check([-A, alpha, L, C], [B, beta, gamma, delta]) == true
```

Where L = IC[0] + sum(pub_i * IC[i+1]), computed via g1_msm (multi-scalar multiplication).

Verifying key (VkData) stored on-chain via set_vk(): contains alpha1 (G1), beta2/gamma2/delta2 (G2), and ic (6 G1 points for 5 public inputs + 1).

7. Claim Contract Logic

File: contracts/zeekpay/src/lib.rs

- **1. Canonical check:** Reject nullifier/root >= BLS12-381 scalar modulus r (prevents n vs n+r aliasing, which would be a dou
- **2. Root check:** Root must exist in contract's persistent storage
- **3. Nullifier check:** Must be unused. Check-then-set, bumped to ~173 day TTL
- **4. Recipient digest check:** Same canonical check as nullifier
- **5. Groth16 verify:** verify(env, vk, proof, [root, nullifier, recipientDigest, amount, tokenId])
- **6. Pay recipient:** token.transfer(contract -> recipient, amount) using SAC address for tokenId
- **7. Emit event:** Only nullifier emitted (no commitment, breaks deposit-claim link)

Token Registration

Token IDs: 0 = USDC, 1 = XLM, 2 = USDT. Each mapped to a Stellar Asset Contract (SAC) address via add_token(token_id, sac_address). Token ID 0 set by initialize(), others via admin-only add_token().

8. Security Properties

Attack	Defense
Double-spend	Nullifier stored persistently, rejected on replay
Non-canonical aliasing	is_canonical_fr() rejects values >= r
Front-running	recipientDigest bound in proof
Amount mismatch	amount bound in commitment + proof
Cross-token drain	tokenId bound in commitment + proof
Fake root	Only admin-posted roots accepted
Fake leaf	Indexer only inserts from confirmed on-chain deposit events

9. Trust Assumptions

- **Off-chain Merkle tree:** The relayer/indexer computes roots honestly. A malicious relayer could insert phantom leaves. M
  only reads confirmed on-chain deposit events.
- **Groth16 trusted setup:** Own ceremony (toxic waste must not leak). Demo-acceptable. MPC ceremony needed for product
- **Poseidon constants:** BN254 constants over BLS12-381 field. Works but not canonical. Replace with native BLS12-38
  production.

10. Architecture Stack

Layer	Tech	Notes
Contracts	Rust + Soroban SDK	zeekpay/ main contract, verifier/ Groth16 verifier
ZK	Circom + snarkjs	Groth16/BLS12-381, in-browser proving via WASM
Backend	Node.js + TypeScript	Resolver, indexer, Merkle tree, Postgres store
Frontend	Next.js + Tailwind	stellar-sdk, Freighter wallet, snarkjs prover
Asset	SAC on Stellar	USDC (id=0), XLM (id=1), USDT (id=2)
Storage	Postgres (Supabase)	Merkle leaves, cursor, user identities

Key File Paths

```
circuits/src/claim.circom            Circom circuit (depth-20 Merkle + nullifier)
contracts/zeekpay/src/lib.rs         Main contract (deposit, claim, post_root)
contracts/zeekpay/src/verifier.rs    Groth16 verifier (BLS12-381 pairing_check)
backend/src/indexer.ts               Deposit event indexer + root poster
backend/src/tree.ts                  Sparse incremental Poseidon-Merkle tree
backend/src/commitment.ts            Commitment + nullifier computation
backend/src/poseidon.ts              Poseidon hash (BLS12-381 field)
frontend/src/lib/prove_browser.ts    Browser-side Groth16 prover
frontend/src/lib/claim_tx.ts         Claim transaction builder
frontend/src/lib/poseidon.ts         Browser Poseidon (mirrors backend)
frontend/src/lib/claim_link.ts       Claim link encode/decode
```